



Polimorfism : Cuprins

- 1. Supraincercarea metodelor (overloading)**
- 2. Suprascrierea metodelor (overriding)**
- 3. Legarea dinamica a metodelor**

Supraîncărcarea - Overloading

- Doua metode din aceeași clasă se numesc supraîncărcate (**overloaded**) dacă:
 - ☐ Au același nume
 - ☐ Diferă prin numărul sau tipul argumentelor
 - ☐ Nu diferă doar prin tipul returnat
- Numele unei metode împreună cu numărul și tipul argumentelor se numește **semnatura** metodei

Overloading

```
public class Number {  
    private double n;  
  
    public Number(double n) {  
        this.n = n; }  
  
    public double getNumber() {  
        return n; }  
  
    public double add(double m)  
    {  
        return n + m; }  
  
    public double add(Number m)  
    {  
        return n + m.getNumber();  
    }  
}
```

- Contextul în care va fi apelată fiecare este determinat automat funcție de parametrii de apel.
- oricând mai pot introduce o nouă metodă ceea ce sporește flexibilitatea programării;
- dacă metodele nu se găsesc în aceeași clasă atunci trebuie să li se atribuie aceași modificatori de vizibilitate.



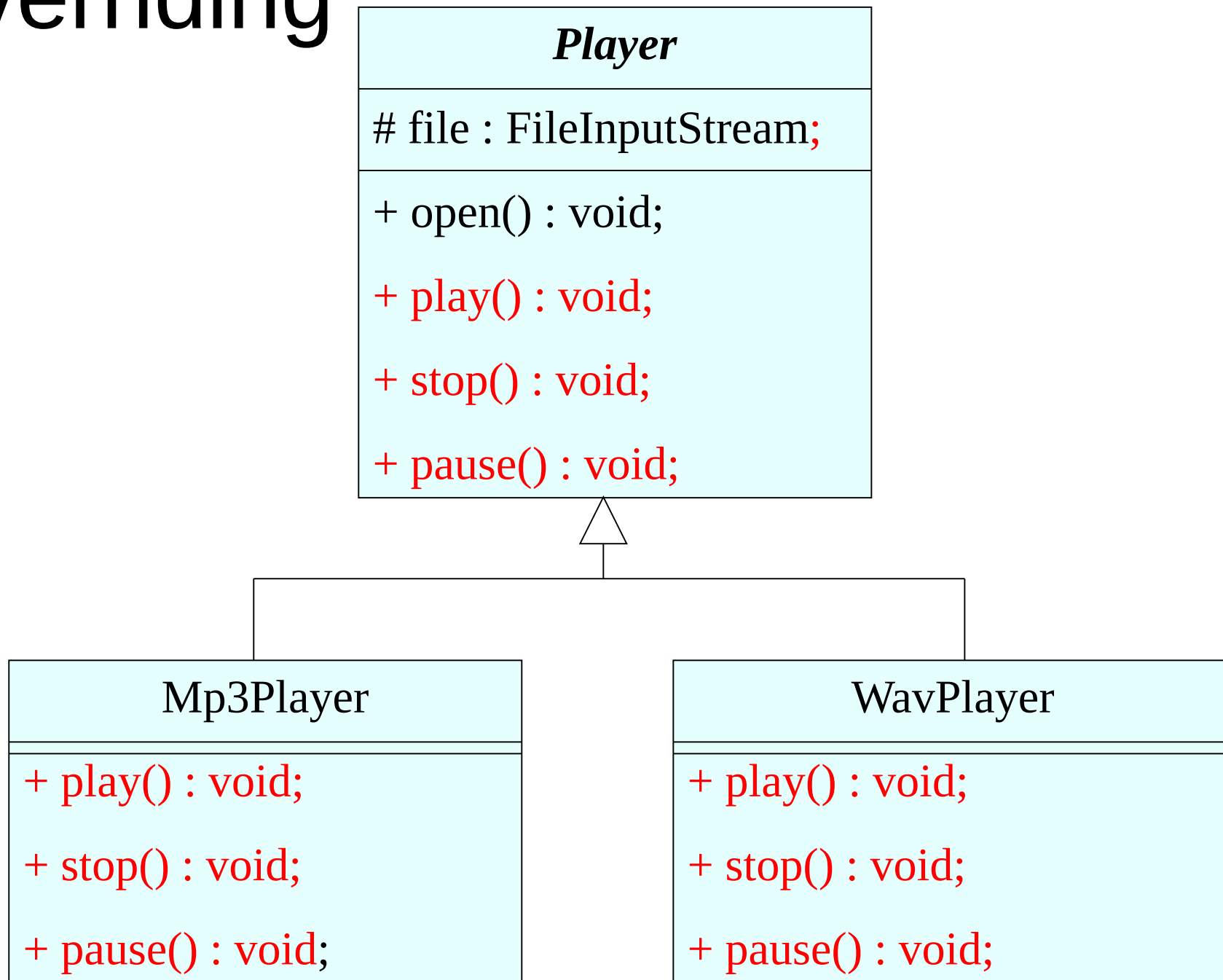
Polimorfism : Cuprins

- 1. Supraincercarea metodelor
(overloading)**
- 2. Suprascrierea metodelor
(overriding)**
- 3. Legarea dinamica a
metodelor**

Overriding

- O metoda dintr-o clasa parinte poate fi suprascrisa (**overridden**) de o metoda din clasa derivata daca:
 - ☐ Au acelasi nume
 - ☐ Au acelasi numar si tip de argumente
 - ☐ Au acelasi tipul returnat
 - ☐ Drepturile de acces nu sunt mai restranse
- O metoda nu poate fi suprascrisa decat o data intr-o clasa

Overriding





Polimorfism : Cuprins

- 1. Supraincercarea metodelor (overloading)**
- 2. Suprascrierea metodelor (overriding)**
- 3. Legarea dinamica a metodelor**

Legarea dinamica

- In cazul unor metode suprascrise (**overridden**) nu se va sti decat la executie care dintre metode va fi apelata
- La apelarea unei metode suprascrise, Java nu verifică tipul referintei ci tipul intern al obiectului pentru care este apelata metoda

Legarea dinamica

```
Player p1 = new Mp3Player();
```

```
Player p2 = new WavPlayer();
```

```
...
```

```
p1.play(); /* Se va apela metoda play() a  
           clasei Mp3Player */
```

```
p2.play(); /* Se va apela metoda play() a  
           clasei WavPlayer */
```



Cuprins

- I. Notiuni de programare orientata obiect**
- II. Organizarea claselor in pachete. Clase fundamentale**
- III. Mostenirea claselor**
- IV. Clase abstracte si interfete**
- V. Polimorfismul**
- VI. Tratarea erorilor**
- VII. Lucrul cu fisiere**
- VIII. Colectii de obiecte**
- IX. Interfete grafice folosind AWT. Applet-uri**
- X. Fire de executie**



Erori : Cuprins

- 1. Tipuri de erori**
- 2. Tratarea erorilor in Java**
- 3. Aruncarea unei exceptii**
- 4. Tratarea unei exceptii**
- 5. Crearea unei exceptii proprii**

Tipuri de erori

■ Erori de compilare

- Sunt detectate si corectate in faza de compilare

■ Erori de executie

- Sunt erori care nu pot fi detectate la compilare: accesarea unor elemente inexistente, fisiere care nu exista, conexiuni de retea gresite, etc.

■ Erori logice

- Programul functioneaza dar nu face ce trebuie
- Nu pot fi detectate decat din functionalitatea programului

Tipuri de erori

- La aparitia unei erori trebuie realizate cel putin urmatoarele lucruri
 - Informarea utilizatorului ca a aparut o eroare
 - Salvarea datelor
 - Permitearea utilizatorului sa iasa din program

Tipuri de erori

- Mecanismele de tratare a erorilor isi propun detectarea si corectarea **erorilor de executie**
- Mecanismele clasice de tratare a erorilor presupun folosirea unor variabile (flag-uri) pentru a testa starea programului.
 - Ex: dupa o operatie cu fisiere daca apare o eroare aceasta este scrisa intr-o variabila ce trebuie testata



Erori : Cuprins

- 1. Tipuri de erori**
- 2. Tratarea erorilor in Java**
- 3. Aruncarea unei exceptii**
- 4. Tratarea unei exceptii**
- 5. Crearea unei exceptii proprii**



Tratarea erorilor in Java

- Mecanismul de tratare a erorilor in Java isi propune sa **separe** codul util de codul de tratare a erorilor, si se realizeaza prin **exceptii**
- **Exceptiile** in Java sunt situatii care apar in timpul executiei unui program si care fac **imposibila** continuarea programului
- Exceptiile sunt **obiecte** care contin informatii despre eroarea aparuta la executie

Tratarea erorilor in Java

- La aparitia unei exceptii:
 - se intrerupe executia normala a programului
 - se creeaza un obiect de tip exceptie
 - se cauta un handler pentru rezolvarea acelei exceptii
 - daca nu se gaseste un handler programul se va opri



Erori : Cuprins

- 1. Tipuri de erori**
- 2. Tratarea erorilor in Java**
- 3. Aruncarea unei exceptii**
- 4. Tratarea unei exceptii**
- 5. Crearea unei exceptii proprii**

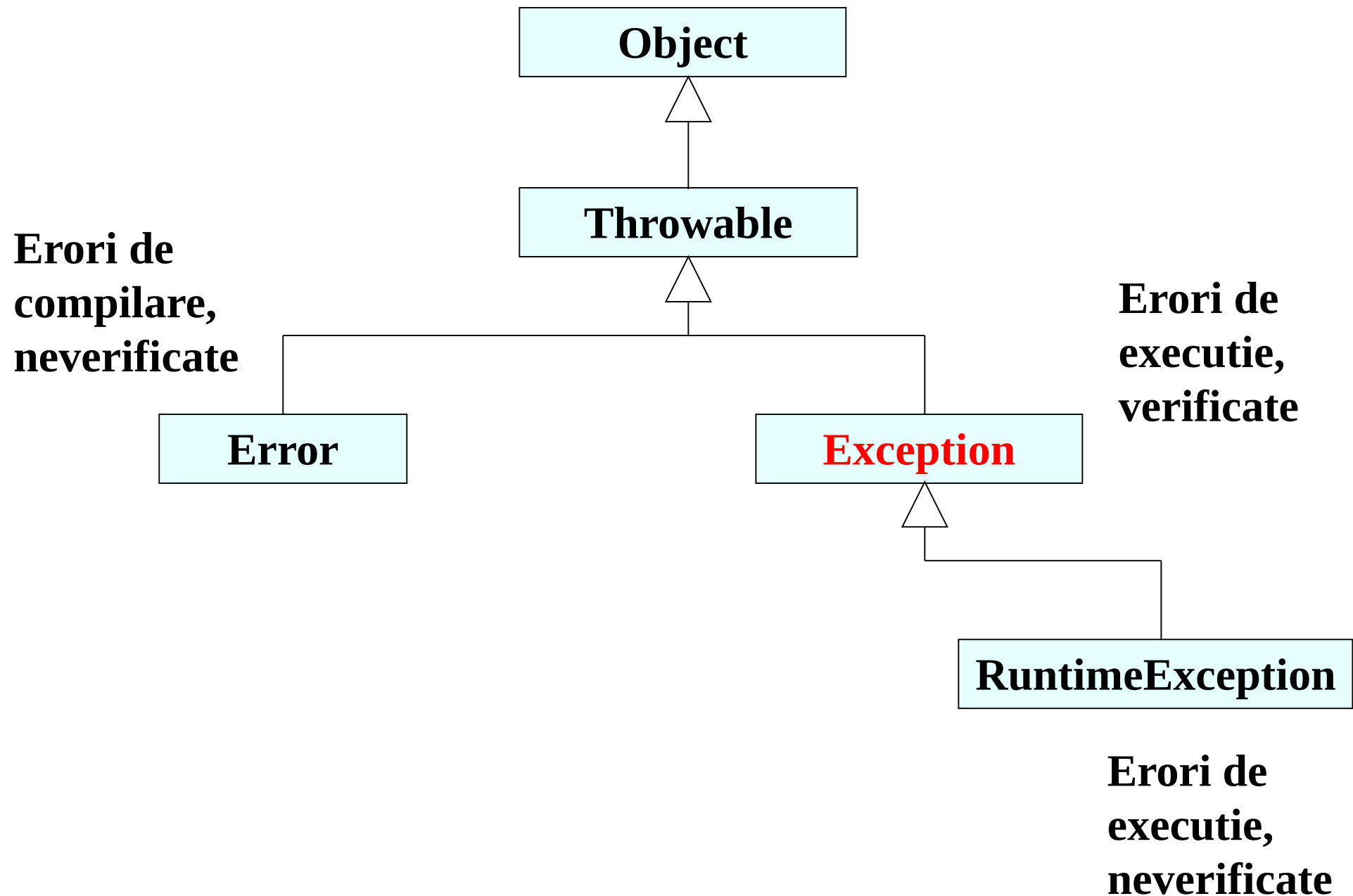
Aruncarea unei exceptii

- Detectarea unei conditii de eroare si generarea unei exceptii se numeste “**aruncarea** unei exceptii” (throw)
- Compilatorul obliga orice metoda care poate genera o exceptie sa **declare** lucrul acesta in definitia ei
- Orice program care foloseste aceasta metoda trebuie **trateze** acea exceptie (handle)

Aruncarea unei exceptii

```
public class ReadFile {  
    private FileInputStream fisier;  
  
    public void open() throws IOException {  
        ...  
        // daca nu exista fisierul  
  
        throw new IOException();  
        ...  
    }  
}
```

Exceptii: Ierarhia de clase





Aruncarea unei exceptii

- Se pot arunca doar obiecte derivate din clasa **Throwable**
- Obiectele derivate din clasa **Error** sunt considerate erori interne si nu se asteapta ca programatorul sa le trateze
- Obiectele derivate din clasa **Exception**, daca sunt aruncate trebuie tratate de programator

Aruncarea unei exceptii

- Obiectele derivate din clasa **RuntimeException** nu trebuie tratate pentru ca sunt exceptii care pot fi rezolvate prin corectarea programului (ex. `ArrayIndexOutOfBoundsException`)
- O metoda care **suprascrie** o metoda ce arunca o exceptie, trebuie si ea sa declare ca arunca acea exceptie sau o exceptie mai concreta, dar nu mai generala



Erori : Cuprins

- 1. Tipuri de erori**
- 2. Tratarea erorilor in Java**
- 3. Aruncarea unei exceptii**
- 4. Tratarea unei exceptii**
- 5. Crearea unei exceptii proprii**

Tratarea unei exceptii

- La aparitia unei exceptii se **opreste** executia normala a programului si se incepe cautarea unui handler pentru tratarea acelei exceptii
- Un **handler** este o bucata de cod care prinde si trateaza o exceptie
- Cautarea unui handler se face intorcandu-ne inapoi in metodele care au apelat metoda in care a aparut exceptia

Tratarea unei exceptii

- O metoda care foloseste cod Java ce poate genera o exceptie, trebuie sa faca unul din urmatoarele lucruri:
 - Sa **declare** la randul ei ca arunca mai departe acea exceptie
 - Sa **prinda** acea exceptie intr-un handler si sa o rezolve
 - Sa prinda acea exceptie intr-un handler si sa o **rearunce** mai departe, declarand ca arunca acea exceptie

Tratarea unei exceptii

```
public class Student {  
    FileInputStream f;  
    public void readFile(String fName) {  
        try {  
            f = new FileInputStream(fName);  
            f.read();  
        } catch (IOException e) {  
            System.out.println("Nu pot citi");  
        } catch (FileNotFoundException e) {  
            System.out.println("Nu exista");  
        }  
    }  
}
```

Tratarea unei exceptii

- Pentru prinderea unei exceptii se include codul care poate arunca o exceptie intr-un **bloc try-catch**
- In blocul **try** este programul normal
- In blocurile **catch** sunt handlerele de exceptii
- Un try trebuie sa aiba cel putin un catch
- Un try poate avea mai multe blocuri catch, cate unul pentru fiecare exceptie ce poate sa apara

Tratarea unei exceptii

- Din cauza polimorfismului, un bloc catch poate prinde orice exceptie derivata din clasa pe care el o declara
- Din aceasta cauza prima data se declara handlerele pentru exceptii mai concrete (mai jos in ierarhie), si apoi se declara handlerele pentru exceptii mai generale

Tratarea unei exceptii

```
public class Student {  
    FileInputStream f;  
    public void readFile(String fName) {  
        try {  
            f = new FileInputStream(fName);  
            f.read();  
        } catch (IOException e) {  
            System.out.println("Nu pot citi");  
        } catch (Exception e) {  
            System.out.println("Nu exista");  
        }  
    }  
}}
```

Tratarea unei exceptii

- Un bloc try-catch poate avea optional un bloc **finally**, care se executa indiferent daca a aparut sau nu o exceptie
- Blocul finally este folosit pentru actiuni care trebuie efectuate indiferent daca a aparut sau nu o exceptie (inchiderea fisierelor, a conexiunilor de retea, etc)
- Singurul caz in care nu se executa blocul finally este atunci cand apare un `System.exit()` inaintea lui

Tratarea unei exceptii

```
public class Student {  
    FileInputStream f;  
  
    public void readFile(String fName) {  
        try {  
            f = new FileInputStream(fName);  
            f.read();  
        } catch (Exception e) {  
            System.out.println("Nu exista");  
        } finally {  
            f.close();  
        }  
    }  
}
```




Erori : Cuprins

- 1. Tipuri de erori**
- 2. Tratarea erorilor in Java**
- 3. Aruncarea unei exceptii**
- 4. Tratarea unei exceptii**
- 5. Crearea unei exceptii proprii**

Crearea unei exceptii proprii

- Pentru definirea unei noi exceptii, tot ceea ce trebuie sa facem este sa cream o clasa proprie derivata din clasa **Exception**
- In continuarea aruncarea, declararea si prinderea acestei exceptii se face ca pentru orice alta exceptie